
TokenCapBench: Forecasting Verified Success Under Token Caps

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Language-model systems often commit inference resources before they know
2 whether a run will succeed: a scheduler chooses which tasks to attempt, a router
3 chooses a model, and a service sets a generated-token cap. Standard benchmarks
4 usually report success at fixed inference settings, while token-use predictors esti-
5 mate consumption rather than verified utility. We introduce **TokenCapBench**, a
6 verifier-backed benchmark for *pre-execution success forecasting under imposed*
7 *token caps*. For each task, model, solver scaffold, verifier, and budget grid, a
8 forecaster predicts the probability that a fresh capped solve will pass. The harness
9 then executes independent solver calls under each cap, verifies every output, and
10 evaluates both probability quality and strict no-overspend allocation. The release
11 contains 1,542 core math and coding tasks with 33,008 verified capped outcomes
12 across four models, plus scoped harder-coding, editing, freshness, and timing
13 diagnostics. Across tracks, success is meaningfully budget-dependent. Raw self-
14 forecasts expose useful budget signal and improve 25% fixed-budget scheduling
15 over always using the maximum cap (150.2 versus 73.2 average core successes),
16 but their probabilities remain miscalibrated: split-safe priors, histogram recal-
17 ibration, and token-use proxy baselines are competitive or stronger for probability
18 quality. TokenCapBench turns resource-aware self-assessment into a reproducible
19 evaluation target for building calibrated budget controllers.

20 1 Introduction

21 A deployed language-model system often has to commit inference resources before observing whether
22 execution succeeds. A serving system may need to route a task to a cheaper or stronger model, cap
23 a generation, defer an expensive request, or choose which tasks to attempt under a shared quota.
24 Existing benchmarks mostly evaluate the downstream outcome: they choose a generation setting, run
25 the model, and report whether the final answer passes. That is necessary for measuring capability, but
26 it does not answer the earlier operational question: *how likely is verified success under each available*
27 *token cap?*

28 This question is distinct from response-length prediction. A short answer can pass, a long answer
29 can fail, and extra tokens can help on one task while doing little on another. It is also distinct from
30 ordinary confidence calibration, because the event is conditional on an imposed resource intervention.
31 The relevant probability is not only $P(\text{success} \mid x)$; it is $P(\text{success} \mid x, m, s, B)$ for task x , model
32 m , solver scaffold s , and generated-token cap B .

33 TokenCapBench makes this probability measurable. The forecasting call sees the task, solver
34 conditions, verifier description, and budget grid, and is asked to output only probabilities. The
35 harness then launches fresh solver contexts under each hard cap, hides the forecast from the solver,
36 verifies every output, and scores both the probabilities and the budget choices induced by those

probabilities. The central object is therefore a verified success curve under controlled token caps, not the token count of an unconstrained trajectory. Figure 1 summarizes this separation between forecasting, budgeted solving, and verification.

This framing is deliberately narrower than a full agent benchmark. The first release uses controlled single-turn solve scaffolds and verifier-backed math and coding substrates. This constraint is a feature: it isolates the budget-conditioned forecasting problem before adding tool calls, retries, repository state, and production serving variability. The same protocol can later be embedded in agent controllers, but the present claims concern pre-execution forecasts under imposed generated-token budgets.

The paper makes four contributions.

1. We define *pre-execution forecasting of verified success*: forecast first, intervene with hard generated-token caps, verify independently, and evaluate both calibration and allocation.
2. We instantiate the protocol on verifier-backed math and coding tasks, including GSM8K, MATH/MATH-500, HumanEval+, MBPP+, BigCodeBench-Hard, CanItEdit, and LiveCodeBench freshness checks.
3. We show that current models have partial budget awareness but weak raw calibration. Simple priors, lightweight recalibrators, and token-usage proxies often match or improve raw self-forecasts.
4. We evaluate forecasts in the decision setting that motivates the benchmark: strict fixed-budget scheduling, where a policy must select task-budget pairs without overspending a global generated-token budget.

2 Related work

Verifier-backed task benchmarks. GSM8K and MATH evaluate mathematical problem solving with answer checks [7, 11]. HumanEval, MBPP, EvalPlus, BigCodeBench, CanItEdit, LiveCodeBench, and SWE-bench evaluate coding, editing, and repository-level capabilities with executable tests or official labels [6, 1, 17, 21, 5, 12, 13]. These benchmarks primarily measure final success under selected inference settings. TokenCapBench overlays a pre-execution forecasting problem on top of such substrates.

Calibration, self-knowledge, and selective prediction. Calibration metrics ask whether predicted probabilities match empirical frequencies [3, 18, 9]. Language-model work studies whether models know what they know and can express uncertainty reliably [14, 16]. Selective prediction studies when a model should answer at all [8]. TokenCapBench changes the event being calibrated: the outcome is verified success under a specified generated-token cap, and the action space is budget selection rather than answer-or-abstain.

Token budgets and token consumption. Prior work studies response-length prediction for serving, token-budget-aware reasoning, adaptive token allocation, and inference-time scaling [20, 10, 15, 4, 19]. Recent agent-token work studies where agentic coding systems spend tokens and whether agents can predict their own unconstrained token usage [2]. TokenCapBench is complementary: its budgets are imposed interventions, and its target is verifier-backed success under those interventions.

Table 1 summarizes the distinction: TokenCapBench evaluates calibrated success curves under imposed token caps, whereas neighboring settings usually fix a budget, predict token use, or adapt spending without separately benchmarking pre-execution success-curve calibration.

3 Benchmark protocol

Let x be a task, m a model, s a solver scaffold, V a verifier, and $\mathcal{B} = \{B_1, \dots, B_K\}$ a grid of hard generated-token budgets. TokenCapBench evaluates $(x, m, s, V, \mathcal{B})$ in three phases.

Forecast. The model sees the task, solver conditions, verifier description, and budget grid. It is asked to output only probabilities, not a solution. The forecast is a structured map $\hat{p}(B)$ estimating the probability that a fresh solver run with generated-token cap B will pass V .

Table 1: TokenCapBench relative to neighboring evaluation problems. The central target is a pre-execution forecast of verifier-backed success under imposed generation budgets.

Setting	Pre-exec.	Verified	Budget curve	Same task	What is missing for our target
Verifier-backed task benchmarks	no	yes	no	yes	Measure solved/not-solved at fixed settings, not forecasts over budget interventions.
Confidence calibration / self-knowledge	yes	mixed	no	yes	Estimate correctness or uncertainty, but not success conditioned on explicit token caps.
Token-use or response-length prediction	yes	no	no	yes	Predict consumption of an unconstrained or served run, not verified success under a controlled cap.
Token-budgeting and test-time compute methods	sometimes	yes	sometimes	yes	Optimize or adapt spending, but do not benchmark pre-execution success-curve calibration.
TokenCapBench	yes	yes	yes	yes	Forecasts and solve attempts are separated; each imposed budget is independently verified.

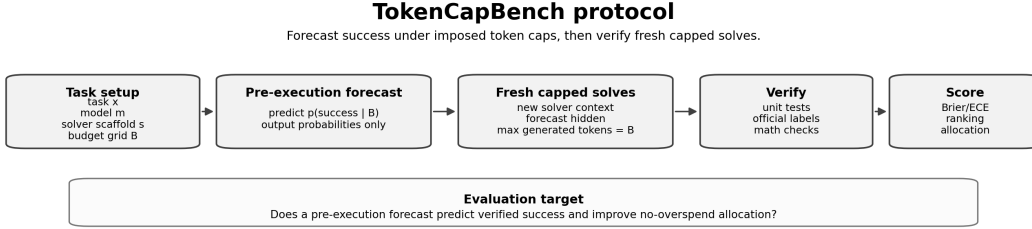


Figure 1: TokenCapBench protocol. A model forecasts a success-by-budget curve before solving. The benchmark then runs fresh capped solves, verifies each output independently, and evaluates both calibration and allocation.

83 **Intervene.** For each $B \in \mathcal{B}$, the harness launches a fresh solver context with cap B . The solver
84 prompt excludes the forecast. This makes the budget an intervention and prevents the forecast from
85 becoming a hidden control signal.

86 **Verify and score.** The verifier maps every budgeted output to a binary label $y(B) \in \{0, 1\}$. The
87 outcome matrix contains one task-budget row per capped solve attempt. Forecast quality is scored
88 with Brier score and expected calibration error (ECE). Allocation quality is scored by the number of
89 verified successes obtained when a policy selects task-budget pairs subject to a fixed global budget.

90 The first budget at which a task passes is the *first-success budget*. Because success can be nonmono-
91 tone under stochastic generation and truncation, the benchmark retains the full success curve rather
92 than forcing monotonicity. Allocation policies are evaluated on the observed matrix, not on smoothed
93 curves.

94 4 Evidence package and metrics

95 Table 2 summarizes the evidence package. The core evidence covers 1,000 math tasks and
96 542 EvalPlus coding tasks across four models, producing 33,008 verified capped solver out-
97 comes. The row-level core outcome artifact is included in the external artifact store and has
98 SHA-256 prefix f2d07c91718e. Extensions test whether the protocol transfers to harder prac-
99 tical coding (BigCodeBench-Hard), lightweight code editing (CanItEdit), and fresher coding tasks
100 (LiveCodeBench-300). We use these extensions with explicit scope: BigCodeBench-Hard is the main
101 hard-coding extension, CanItEdit is a provided-test editing bridge, and LiveCodeBench-300 is an
102 appendix freshness check rather than a standalone main claim.

103 **Forecast metrics.** For each task-budget row i , Brier score is

$$\frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2.$$

Table 2: Evidence scope. Core claims use math and EvalPlus coding. Extension and diagnostic tracks test transfer and resource logging, but are scoped separately from the main claims.

Role	Track	Tasks	Models	Outcome rows	Verifier / labels	Use in paper
Core	GSM8K + MATH	1000	4	20000	numeric / symbolic	main calibration and allocation claims
Core	HumanEval+ + MBPP+	542	4	13008	EvalPlus unit tests	main calibration and allocation claims
Extension	BigCodeBench-Hard	148	5	4440	package labels	hard practical coding extension
Extension	CanItEdit	105	4	2100	provided tests	code-editing bridge
Appendix	LiveCodeBench-300	300	2	3600	official labels	freshness check
Diagnostic	timing/SLA subset	100	2	3600	same verifiers as core	resource logging only

Table 3: Core-track summary regenerated from canonical row-level artifacts. Ranges are across models; baseline columns report held-out mean Brier across models. Lower Brier is better.

Suite	Models	Success@max	Raw Brier range	Rank acc. range	Raw Brier	Recal. Brier	Prior Brier
HumanEval+	4	0.512-0.854	0.153-0.324	0.468-0.527	0.205	0.165	0.182
MBPP+	4	0.000-0.664	0.299-0.460	0.276-0.476	0.256	0.191	0.222
Math core	4	0.529-0.869	0.127-0.255	0.564-0.798	0.184	0.151	0.174

ECE bins forecast probabilities and compares average confidence with empirical success frequency in each bin. We also report success at the maximum budget to separate solver capability from forecast quality. First-success-budget error, underbudgeting, overbudgeting, pairwise ranking accuracy, and regret diagnose how forecasts order tasks and budgets.

Operational allocation. The motivating decision is not just whether probabilities are calibrated; it is whether they support better budget choices. In strict fixed-budget scheduling, a policy selects at most one budget per task and must satisfy $\sum_i B_i \leq C$ for a global generated-token budget C . Overspending is disallowed. We compare raw self-forecasts with empirical priors, recalibrators, token-usage proxies, random policies, max-budget policies, and oracle policies.

Resource accounting. The controlled intervention in the present paper is the generated-token cap. The harness also logs prompt tokens, completion tokens, total visible tokens, provider-exposed reasoning-token fields, finish reason, cap-hit/truncation flags, retry counts, and generation, verification, and end-to-end wall-clock timing when available. We treat timing as an appendix/SLA extension in this release because latency is noisier than token caps and requires repeated, controlled serving conditions to support main-paper claims.

5 Results

5.1 Token budgets materially change verified success

Figure 2 shows nonflat success curves across generated-token caps. This supports the benchmark target: if success were nearly constant in budget, budget-conditioned forecasting would collapse to ordinary task success measurement. Instead, budget effects differ across task sources and models. Lines show canonical row-level success rates; bootstrap uncertainty summaries are included in the artifact reports.

5.2 Raw forecasts have signal, but calibration is not solved

Table 3 shows partial budget awareness: forecast ranking and calibration vary meaningfully across suites and models, with MBPP+ exposing a particularly difficult near-all-failure regime for several models. This matters because a budget controller needs to rank both tasks and budgets before spending tokens. Figure 3 makes the stronger conclusion clear: raw self-forecasts are not enough. Calibration-split priors and recalibrators often improve Brier score over raw forecasts. Models therefore expose budget-relevant information, but direct self-reports are not yet reliable probability estimates.

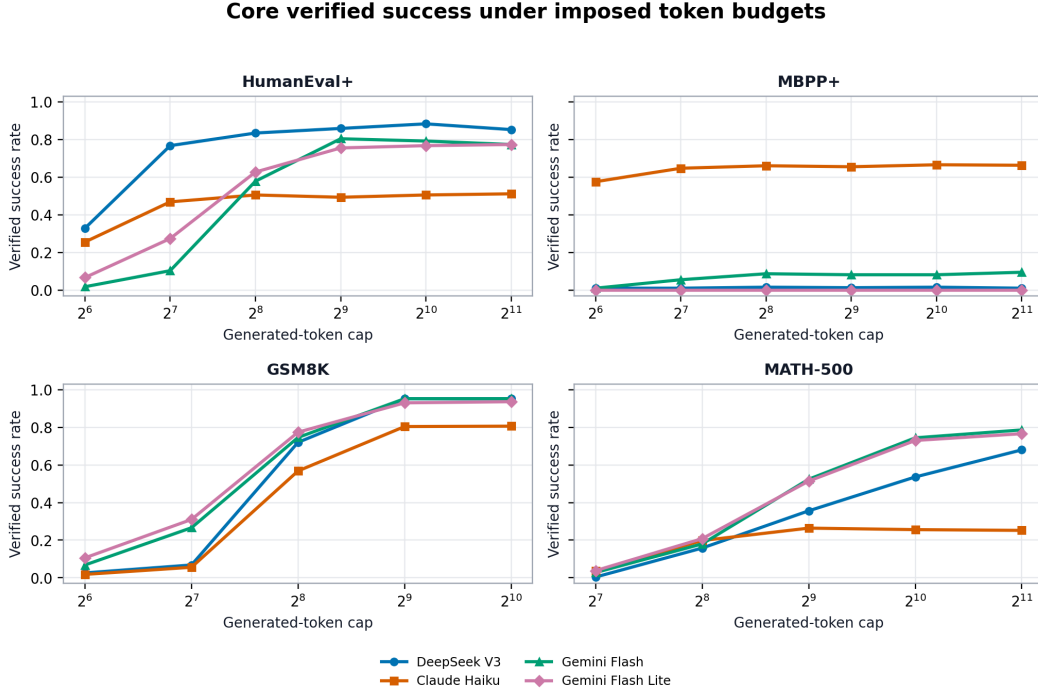


Figure 2: Verified success under imposed generated-token budgets on the core tracks. The math core is split into GSM8K and MATH-500 panels, and EvalPlus coding is split into HumanEval+ and MBPP+ panels. Confidence bands are omitted; lines show canonical row-level success rates.

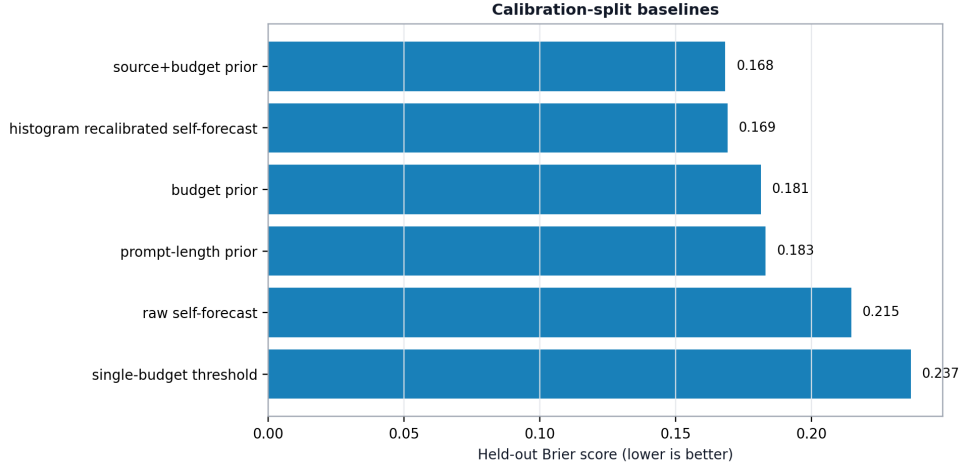


Figure 3: Calibration-split baseline comparison on held-out evaluation tasks. Raw self-forecasts contain signal, but empirical priors and recalibrated forecasts frequently match or improve on them. Lower Brier score is better.

134 5.3 Forecasts matter when they improve allocation

135 Strict scheduling turns calibration into a decision problem. At a fixed global token budget, a scheduler
 136 must choose which task-budget pairs to run. Figure 4 and Table 4 show this evaluation with zero
 137 overspend violations. On the core tracks at the 25 percent budget fraction, raw forecasts obtain 150.2
 138 average verified successes, compared with 73.2 for always using the maximum cap and 137.2 for the
 139 best simple prior. This is the decision-facing comparison the benchmark is designed to support: a
 140 probability forecast is useful only if it improves verified utility under a real budget constraint.

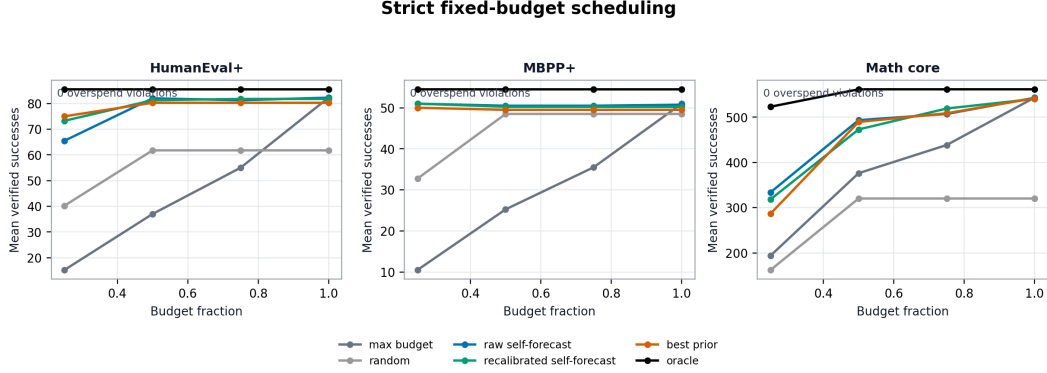


Figure 4: Strict fixed-budget scheduling on the core tracks. Policies select task-budget pairs subject to the same global generated-token budget; the regenerated allocation table reports zero overspend violations.

Table 4: Strict fixed-budget scheduling at the 25 percent core budget fraction. Rows are unweighted averages over core suite summaries and enforce zero overspend violations.

Policy	Avg. successes	Avg. regret to oracle	Avg. budget slack	Overspend viol.
max budget	73.2	147.8	31659	0
random	78.4	142.7	31147	0
raw self-forecast	150.2	70.9	31147	0
recalibrated self-forecast	147.6	73.5	47563	0
best prior	137.2	83.8	49600	0
oracle	221.1	0.0	80763	0

5.4 Hard coding and editing extensions are useful, but should be scoped carefully

BigCodeBench-Hard extends the protocol to more practical Python generation with official package labels. CanItEdit provides a lightweight editing bridge with provided-test verification. Table 5 and Figure 5 show that the same forecast-then-cap-then-verify protocol applies beyond HumanEval and MBPP. We do not present CanItEdit as a definitive hidden-test editing benchmark; its value here is to show that budget-conditioned forecasting naturally applies to edit-style prompts.

5.5 Token usage is a strong feature, not the benchmark target

Token-usage forecasts and proxies can be highly informative. Table 6 shows that token-proxy baselines can outperform raw success forecasts after being mapped to verified outcomes on calibration tasks. This does not make TokenCapBench redundant. Token-use prediction estimates resource consumption; TokenCapBench evaluates the reduced utility available under imposed caps. A strong practical controller should use token features, task features, historical priors, and raw self-forecasts together, then be evaluated by calibrated success and allocation performance.

6 Discussion

TokenCapBench is strongest when framed as a benchmark for *budgeted decision-making*, not as a generic token-consumption benchmark. The main empirical lesson is not that current models are hopeless or that raw self-forecasts are useless. The lesson is sharper: current models reveal partial information about budget-conditioned success, but simple baselines remain strong, and the practical value of a forecast only appears when it improves no-overspend allocation under a fixed budget.

The next version should add multi-resource success surfaces rather than replace the token-budget protocol. A deployment-oriented extension should evaluate success under service-level objectives: generated-token cap, total-token cap, wall-clock latency, retry budget, and provider-error budget. These should be reported as secondary resource metrics unless they are collected under repeated,

Table 5: Practical coding extensions. BigCodeBench-Hard uses official package labels; CanItEdit uses provided-test verification and is framed as an editing bridge.

Track	Model	Tasks	Success@max	Brier	ECE	Rank acc.
BigCodeBench-Hard	DeepSeek V3	148	0.115	0.398	0.494	0.609
BigCodeBench-Hard	Claude Haiku	148	0.101	0.430	0.533	0.452
BigCodeBench-Hard	Gemini Flash	148	0.000	0.436	0.565	—
BigCodeBench-Hard	Gemini Flash Lite	148	0.000	0.392	0.507	—
BigCodeBench-Hard	GPT-5-mini	148	0.331	0.469	0.559	0.577
CanItEdit	DeepSeek V3	105	0.676	0.211	0.108	0.481
CanItEdit	Claude Haiku	105	0.581	0.298	0.288	0.431
CanItEdit	Gemini Flash	105	0.629	0.253	0.237	0.423
CanItEdit	Gemini Flash Lite	105	0.667	0.225	0.148	0.407

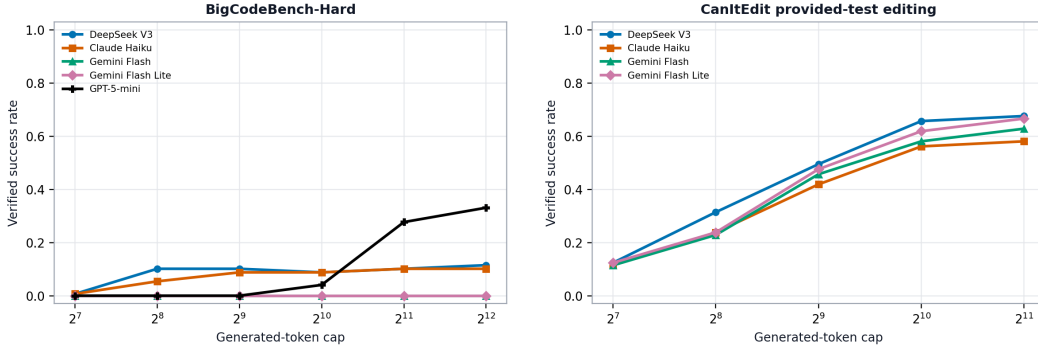


Figure 5: Budget-success curves for practical coding extensions. Left: BigCodeBench-Hard with official package labels. Right: CanItEdit provided-test-only editing bridge.

164 controlled serving conditions. The target then becomes

$$P(\text{success} \mid x, m, s, \text{generated tokens}, \text{total tokens}, \text{latency}, \text{retries}),$$

165 with allocation evaluated on the same verified outcomes.

166 7 Limitations

167 The present release studies controlled single-turn solve scaffolds. It does not claim to evaluate
 168 full production agents, hidden reasoning compute, or Docker-heavy repository repair. CanItEdit
 169 uses provided tests, so we frame it as an editing bridge rather than a hidden-test editing result.
 170 LiveCodeBench-300 uses two models and remains a freshness check. Public benchmark contamination
 171 is possible on established datasets. Verifiers are imperfect: code labels inherit test-suite coverage,
 172 and math labels depend on answer extraction and symbolic/numeric matching.

173 Token counts are provider- and tokenizer-dependent. Generated-token caps do not measure all hidden
 174 inference compute, and wall-clock timing can vary with provider load, hardware, batching, and
 175 retry behavior. For that reason, timing is logged but not used for main claims in this release. These
 176 limitations define the first-release boundary and motivate the next benchmark version rather than
 177 invalidating the core protocol.

178 8 Broader impact

179 Budget-aware forecasting can make model use more transparent by exposing the expected success
 180 tradeoff of additional inference resources. It can also be misused if noisy forecasts are treated as
 181 guarantees or used to ration access without human oversight. We therefore frame TokenCapBench as
 182 an evaluation tool for calibrated decision support, not as an automatic policy for accepting, rejecting,
 183 or pricing user requests.

Table 6: Token-usage proxy comparison. Token-use predictions can be useful auxiliary features, but they are evaluated here only after being mapped to verified success under imposed budgets.

Suite	Usage coverage	Success-rank acc.	Token-proxy rank acc.	Raw success Brier	Token-proxy Brier
BigCodeBench-Hard	1.00	0.525	0.583	0.427	0.043
HumanEval+	1.00	0.568	0.642	0.222	0.193
MBPP+	1.00	0.421	0.573	0.401	0.075
Math core	1.00	0.704	0.729	0.170	0.165

Table 7: Repository validation summary after restoring the canonical core outcome artifact and regenerating the paper tables and figures.

Check	Result
Core outcome JSONL rows	PASS (33,008)
Core outcome SHA-256	PASS (prefix <code>f2d07c91718e</code>)
Core forecast rows	PASS (6,168)
HumanEval+ outcome rows	PASS (3,936)
MBPP+ outcome rows	PASS (9,072)
Math core outcome rows	PASS (20,000)
Paper tables/figures regenerated	PASS
Supplementary release archive	PASS

184 9 Reproducibility and release status

185 The repository includes task snapshots, prompts, forecast and outcome schemas, metric code, compact
 186 tables and figures, run manifests, dataset/evaluation cards, and validation scripts. Table 7 gives the
 187 current validation summary after restoring the canonical row-level core outcome artifact and regener-
 188 ating the paper tables and figures. The paper source uses the supplied NeurIPS 2026 Evaluations &
 189 Datasets style file. The artifact is packaged as a local release archive for anonymous supplementary
 190 submission; a stable hosted URL can be added in the submission form when assigned.

191 10 Conclusion

192 TokenCapBench asks a concrete question that ordinary task benchmarks and token-consumption
 193 predictors do not answer: before execution, how likely is verified success under each imposed token
 194 cap? The forecast-then-intervene protocol separates self-assessment from solving, imposes hard
 195 generated-token caps, verifies every outcome, and evaluates both calibration and allocation. The
 196 current evidence shows that budget-conditioned success is real, raw self-forecasts are informative but
 197 miscalibrated, and strict fixed-budget scheduling exposes the operational effect of those forecasts.
 198 This creates a clear target for future forecasters and controllers: choose budgets before execution with
 199 calibrated expectations of verifier-backed success.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program Synthesis with Large Language Models, 2021.
- [2] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks, 2026.
- [3] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Re, and Azalia Mirhoseini. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024.
- [5] Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Jacob Ginesin, Edward Berman, George Chakraborty, Anton Lozhkov, Carolyn Jane Anderson, and Arjun Guha. CanItEdit: A 105-problem benchmark that tests whether LLMs can edit code from natural-language instructions, 2023.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models Trained on Code, 2021.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems, 2021.
- [8] Yonatan Geifman and Ran El-Yaniv. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, 2017.
- [9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning*, 2017.
- [10] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-Budget-Aware LLM Reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24842–24855, 2025.
- [11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving with the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [12] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *International Conference on Learning Representations*, 2025.
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*, 2024.
- [14] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language Models (Mostly) Know What They Know, 2022.
- [15] Zheng Li, Qingxiu Dong, Jingyuan Ma, Di Zhang, and Zhifang Sui. SelfBudgeter: Adaptive Token Allocation for Efficient LLM Reasoning, 2025.
- [16] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching Models to Express Their Uncertainty in Words. *Transactions on Machine Learning Research*, 2022.

- 247 [17] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by
248 ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation,
249 2023.
- 250 [18] Alexandru Niculescu-Mizil and Rich Caruana. Predicting Good Probabilities with Supervised
251 Learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages
252 625–632, 2005.
- 253 [19] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute
254 Optimally can be More Effective than Scaling Model Parameters, 2024.
- 255 [20] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response
256 Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline.
257 In *Advances in Neural Information Processing Systems*, 2023.
- 258 [21] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam
259 Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. BigCodeBench: Benchmarking
260 Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint*
261 *arXiv:2406.15877*, 2024.

A Additional implementation details

The benchmark logs generated-token budget, finish reason, cap-hit/truncation flag, prompt tokens, completion tokens, total visible tokens, provider-exposed reasoning-token fields, retry count, generation wall time, verification wall time, and end-to-end wall time. The main results use generated-token caps because they are the controlled intervention shared across the current tracks. The logged timing, retry, and total-token fields are intended for SLA-style extensions and resource-accounting diagnostics.

Recommended secondary metrics for the artifact are log score after probability clipping, bootstrap confidence intervals over tasks, cap-hit rate, retry rate, p50/p95 generation latency, p50/p95 verification latency, end-to-end latency, and throughput in tokens per second. These metrics should be reported in the appendix or artifact card unless the experiment uses repeated measurements on stable serving infrastructure.

B Checklist

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#).

Justification: The abstract and introduction state the benchmark protocol, the verifier-backed evidence scope, and the limits of raw self-forecast calibration.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#).

Justification: The Limitations section covers public benchmark contamination, verifier coverage, hidden reasoning compute, provider/tokenizer cap dependence, and timing variability.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

Answer: [\[N/A\]](#).

Justification: The paper introduces and evaluates a benchmark protocol; it does not claim new theoretical results.

4. Experimental result reproducibility

Question: Does the paper disclose the information needed to reproduce the main experimental results?

Answer: [\[Yes\]](#).

Justification: The repository includes task snapshots, prompts, schemas, canonical row-level forecast/outcome artifacts, metric code, and regeneration scripts.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to reproduce the main experimental results?

Answer: [\[Yes\]](#).

Justification: The release archive contains the code, compact tables and figures, external artifact manifest, Croissant metadata, and canonical row-level artifacts. Anonymous hosted URL assignment remains author-owned.

6. Experimental setting/details

Question: Does the paper specify the experimental setting details necessary to understand the results?

309 Answer: [Yes].

310 Justification: The protocol, budget grids, forecast/solve separation, verifier-backed task

311 sources, calibration split, and strict allocation setting are described in the paper and artifact

312 docs.

313 **7. Experiment statistical significance**

314 Question: Does the paper report error bars or other appropriate information about statistical

315 significance?

316 Answer: [Yes].

317 Justification: Tables report bootstrap intervals where available. The regenerated Figure

318 2 caption explicitly states that confidence bands are omitted for direct row-level success

319 curves.

320 **8. Experiments compute resources**

321 Question: Does the paper provide sufficient information on compute resources needed to

322 reproduce the experiments?

323 Answer: [Yes].

324 Justification: The artifact logs generated-token caps, token usage fields, retry counts, wall-

325 clock timing, and cost/runtime summaries; timing is scoped as diagnostic rather than a

326 provider benchmark.

327 **9. Code of ethics**

328 Question: Does the research conform with the NeurIPS Code of Ethics?

329 Answer: [Yes].

330 Justification: The work evaluates benchmark artifacts and model outputs on public/verifier-

331 backed tasks without human subjects or sensitive personal data collection.

332 **10. Broader impacts**

333 Question: Does the paper discuss positive and negative societal impacts?

334 Answer: [Yes].

335 Justification: The Broader impact section discusses transparent budget-aware decision

336 support and the risk of treating noisy forecasts as guarantees.

337 **11. Safeguards**

338 Question: Does the paper describe safeguards for responsible release of high-risk data or

339 models?

340 Answer: [N/A].

341 Justification: The paper releases benchmark artifacts and code, not a model or high-risk

342 scraped dataset; release metadata and secret-scrub checks are included.

343 **12. Licenses for existing assets**

344 Question: Are existing assets properly credited and are license and terms of use mentioned?

345 Answer: [Yes].

346 Justification: The paper cites the underlying benchmark sources and the repository includes

347 license, citation, dataset card, and evaluation card files.

348 **13. New assets**

349 Question: Are new assets introduced in the paper documented and provided alongside the

350 assets?

351 Answer: [Yes].

352 Justification: TokenCapBench artifacts are documented through README, MANIFEST,

353 Croissant metadata, dataset/evaluation cards, and release manifests.

354 **14. Crowdsourcing and research with human subjects**

355 Question: For crowdsourcing experiments and research with human subjects, does the paper

356 include the required participant information?

357 Answer: [N/A].
358 Justification: The work does not involve crowdsourcing experiments or human subjects
359 research.
360 **15. Institutional review board approvals or equivalent for research with human subjects**
361 Question: Does the paper describe risks, disclosures, and approvals for human subjects
362 research?
363 Answer: [N/A].
364 Justification: The work does not involve human subjects research.
365 **16. Declaration of LLM usage**
366 Question: Does the paper describe LLM usage if it is an important or non-standard compo-
367 nent of the research?
368 Answer: [Yes].
369 Justification: LLMs are the forecast and solver subjects of the benchmark, and the protocol
370 describes their role and separation.